BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN ENGLISH

DIPLOMA THESIS

An Actor-Critic Approach for the Recommendation of Algorithmic Programming Questions

Supervisor
Lecturer, PhD Bocicor Maria Iuliana

Author
Dan Laurentiu

2025

ABSTRACT

Creating an effective LeetCode study plan has many significant challenges for learners at all skill levels. With over 3,500 problems divided into 71 different topics and complex prerequisite relationships, even experienced programmers will struggle to create optimal learning plans. Traditional approaches rely on community-curated study plans, which cannot possibly adapt to individual learners. This work addresses these limitations by introducing an adaptive recommendation system powered by an Actor-Critic reinforcement learning algorithm.

We present our actor-critic model that learns to recommend personalized sequences of LeetCode problems. These recommendations are crucially adjusted to each user's evolving skill level.

The practical implementation of this thesis is demonstrated through a2cl, a web application. The application features a frontend interface for tracking progress and visualizing learning plans, along with a backend server that hosts our RL environments. Both of these servers are deployed in a production environment.

Our work provides proof that learning systems can be successfully deployed in practical tools. While our results show desirable convergence samples, with reward metrics improving from 100 to over 400 (moving average) during training, there are many areas for improvement. Gradient instability in the actor has been a constant challenge. Also, significant imbalance is present in topic coverage, where popular subjects had orders of magnitude more problems (1000+) compared to niche topics (under 10).

Contents

1 Introduction **1**

2 LeetCode in Technical Interviews: Challenges and Opportunities **2**
2.1 Introduction to Algorithmic Programming Problems 2
2.2 Challenges with Manually Structuring Plans 3
2.3 Analysis of Neetcode.io 3

3 Reinforcement Learning and Planning Strategies **5**
3.1 The Domain of Reinforcement Learning 5
3.2 Elements of Reinforcement Learning 7
3.2.1 Evaluating Values and Policies 10
3.2.2 Improving Values and Policies 11
3.3 Advantage Actor-Critic Method 12
3.3.1 Motivations 12
3.3.2 Formal Presentation 12

4 Literature survey **15**
4.1 Deep-Q-Learning Methods 15
4.2 Model-Based Methods 18

5 a2cl: Web Application for Tracking Progress **20**
5.1 Model Development 20
5.1.1 Data Analysis 20
5.1.2 Data Collection and Pre-Processing 23
5.1.3 Development of the Environment 27
5.1.4 Hyperparameter Tuning 29
5.1.5 Results 32
5.2 Application Development 33
5.2.1 Feature Specification 33
5.2.2 Architectural Overview 36
5.2.3 Frontend Architecture and User Experience 37

5.2.4 Backend Architecture and Data Flow 37
5.2.5 Security, Validation and Testing 39

6 Conclusion and Future Work **41**

Bibliography **43**

Chapter 1

Introduction

With the rise in popularity of algorithmic programming problems proposed in technical interviews for software engineering roles, developing a practical but effective learning plan is crucial. Building a system with these two values in mind is not intuitive, and only people with both broad and deep experience can be expected to be efficient at it. However, this makes the problem counterintuitive, and, to the best of our knowledge, there is currently no automated tool to help with this.

This thesis aims to address the gap by proposing an Advantage Actor-Critic model (A2C) [MBM+16], with the purpose of generating learning plans specifically for LeetCode [Lee15] problems. This model will be integrated into the backend of a web application, serving as a proof-of-concept. The application, a2cl, will allow for creating, monitoring, and tracking the learning process.

Our work presents two main contributions. Firstly, it introduces a web-based tool designed to generate sequences of coding problems. Secondly, it advances the application of deep reinforcement learning by implementing the Advantage Actor-Critic algorithm for this specific task. Also, the work includes data analysis of LeetCode problems, offering information that may inform future research.

This thesis will be organized into six chapters. Following this introduction, Chapter 2 provides conceptual knowledge about LeetCode problems and algorithmic learning and presents real challenges learners face. Chapter 3 introduces the reinforcement learning theory necessary for understanding the model. After this, Chapter 4 reviews existing literature on similar reinforcement learning planners. Section 5.1 of Chapter 5 presents in detail all iterations of the proposed solution, together with the final results. Section 5.2 examines the development of the application. Finally, Chapter 6 concludes with problems faced, limitations, and future research directions that would improve upon the work presented.

This work fully represents my own efforts and contributions. No generative AI tools were used in the creation of this thesis or in the development of my solutions.

Chapter 2

LeetCode in Technical Interviews: Challenges and Opportunities

This chapter establishes foundational knowledge about what algorithmic programming problems are and how they are structured on LeetCode, introducing some terminology. Also, it emphasizes the complex dynamics that go into planning a sequence of problems to solve. Lastly, the need for structure is demonstrated with an analysis of the most popular website that provides plans for learning LeetCode problems called NeetCode.io [Nee22].

2.1 Introduction to Algorithmic Programming Problems

Algorithmic programming questions are timed problem-solving tasks that require candidates to write code. These questions usually look for the implementation of efficient solutions, using favorable data structures and algorithms.

On LeetCode, each coding problem is assigned an identifier (from 1 to 3585)[1] and a title. All problems need to be solved with some specific constraints in mind. Because of this, each exercise can be done in many different ways, and each problem has many topic tags associated with it. Some common constraints are: boundaries for input size and time complexity [ALP16].

The LeetCode platform categorizes programming problems into four primary domains, each representing a crucially indistinct aspect of problem-solving. These categories are:

- **Data Structures** - refer to formats for organizing, storing and manipulating data. Common examples include: hash tables, arrays and graphs.

[1] Last checked: June 2025

- **Algorithms** - are systematic rules designed to solve specific problems. This category includes fundamental techniques such as breadth-first search, two pointers and binary search.
- **Concepts** - represent abstract principles underlying programming problems. For the most part, each algorithm belongs to a foundational concept. Dynamic programming, greedy and backtracking are very popular examples of LeetCode concepts.
- **Miscellaneous** - encompasses specialized problem types that do not fit into the above categories. Examples include design challenges, concurrency and game theory problems.

2.2 Challenges with Manually Structuring Plans

A critical challenge in manually designing algorithmic problem-solving plans lies in the graph-like dependency structure between concepts. Algorithms and data structures form a complex prerequisite network where many fundamental algorithms are building blocks for other, more advanced techniques. With each new topic added, this relationship's size exponentially increases and becomes too complex to be captured by hand.

This is worsened by the uneven level of relevance in the algorithms. While certain techniques appear in many questions, others are niche and rarely tested. This means that focusing on crucial problems at the right stage of learning is way more significant than just volume. Also, the scale of the problem space (with over 3,500 problems on LeetCode and growing) makes manual curriculum design very impractical.

Another important challenge is called the "beginner's paradox". In order to create an effective study plan, tons of prior knowledge is needed. However, gaining that knowledge in the first place requires following a well-organized plan over a long period of time. This creates a problem for learners, who don't know where to start or how to structure their learning effectively.

2.3 Analysis of Neetcode.io

Neetcode.io is a very popular website where lists of must-do problems, created by a single developer, are hosted. Neetcode.io filters out the noise by creating plans

75, 150, and 250 problems long. These exercises are grouped into topics and structured in a tree, providing that desired order. This highlights the key problem with LeetCode: its problem set is too large and too unstructured.

However, the learning method used by Neetcode.io has some major drawbacks. The selection of 75 to 250 problems makes learning manageable, but it also leaves out recently added problems. Manually updating plans means that new topics are probably not going to be added in time. Because of this, the content will become outdated, especially as technical interviews change and new problem types become more or less common.

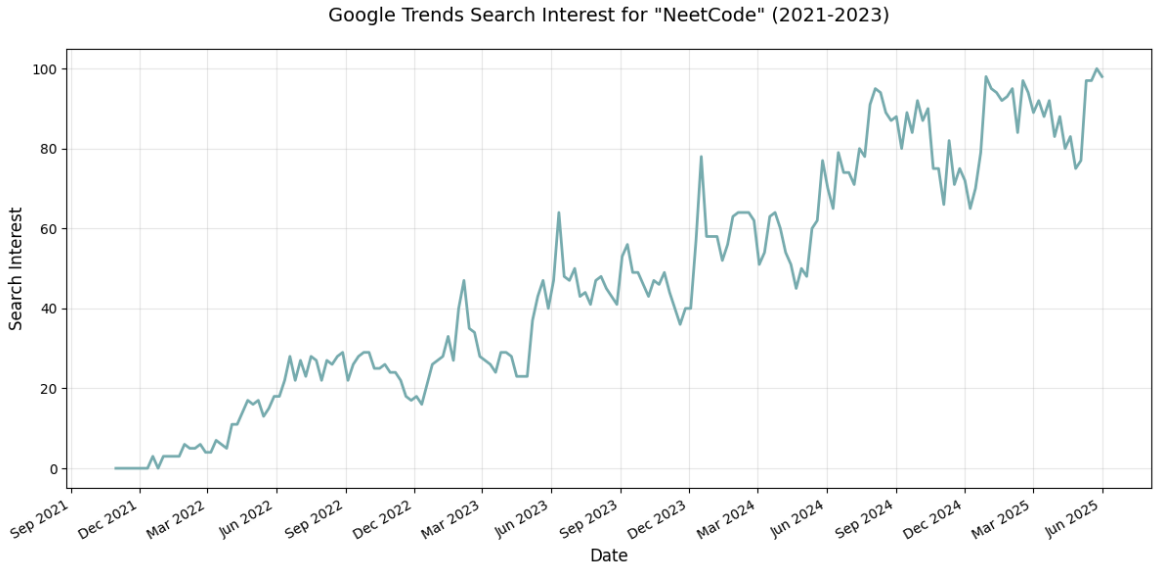


Figure 2.1: Google Trends [Goo23] search interest for ”NeetCode”. The peak interest occurred in May 2025.

Chapter 3

Reinforcement Learning and Planning Strategies

This chapter aims to address Reinforcement Learning concepts that are to be used in this work. In the first section, fundamental high-level concepts will be explained. In the second section, formal mathematical definitions and terminology will be presented. Additionally, in the last section, the general algorithm used for this thesis is going to be explored as well.

3.1 The Domain of Reinforcement Learning

Machine learning techniques are usually included in one of these three primary branches: supervised, unsupervised, and reinforcement learning. Supervised learning trains models on labeled datasets, where each input is paired with a corresponding output. The algorithm learns to map inputs to outputs, which should generalize on unseen data. Crucially, the correctness of these outputs can be rated using the labels. These labels represent the absolute truth. For example, email spam filters use supervised learning to classify messages as ”spam” or ”not spam” based on previously experienced events which are turned into labeled data. In contrast, unsupervised learning processes unlabeled data to uncover hidden structures within it. These structures are not predefined. They are usually groupings or associations. For instance, clustering customers together based on previous purchasing behavior.

The approach we aim to explore in this chapter, called reinforcement learning, is focused on goal-directed learning from interaction. In this branch, an agent learns optimal behaviors through trial-and-error interactions with the environment. The term ”reinforcement” originates from behavioral psychology [Ski53], referring to rewards (positive or negative) that make certain actions more or less likely to be repeated. In Reinforcement Learning (RL), the agent receives rewards for desirable

actions and penalties for errors. Using these signals, an agent can refine its strategy, over millions of steps, in order to maximize cumulative rewards.

Unlike supervised learning, RL does not have labeled data. Instead, it relies on feedback from the environment. This makes the reward signal a kind of indirect supervisor. However, unlike supervised learning, rewards are not absolute or objectively "correct". These are designed by humans and contain biases and assumptions about the task. Another crucial difference, still related to the reward, is its delayed nature. RL is able to solve problems even when the reward is sparse, meaning that it is only acquired sometime in the future. Additionally, each action made by an agent will change the state of the environment and subsequently impact all future actions. Essentially, this is what makes reinforcement learning fundamentally different from the other types and is what makes it a "closed-loop" problem.

The typical flow [Sil15] of a reinforcement learning interaction process between the agent and the environment follows the diagram below.

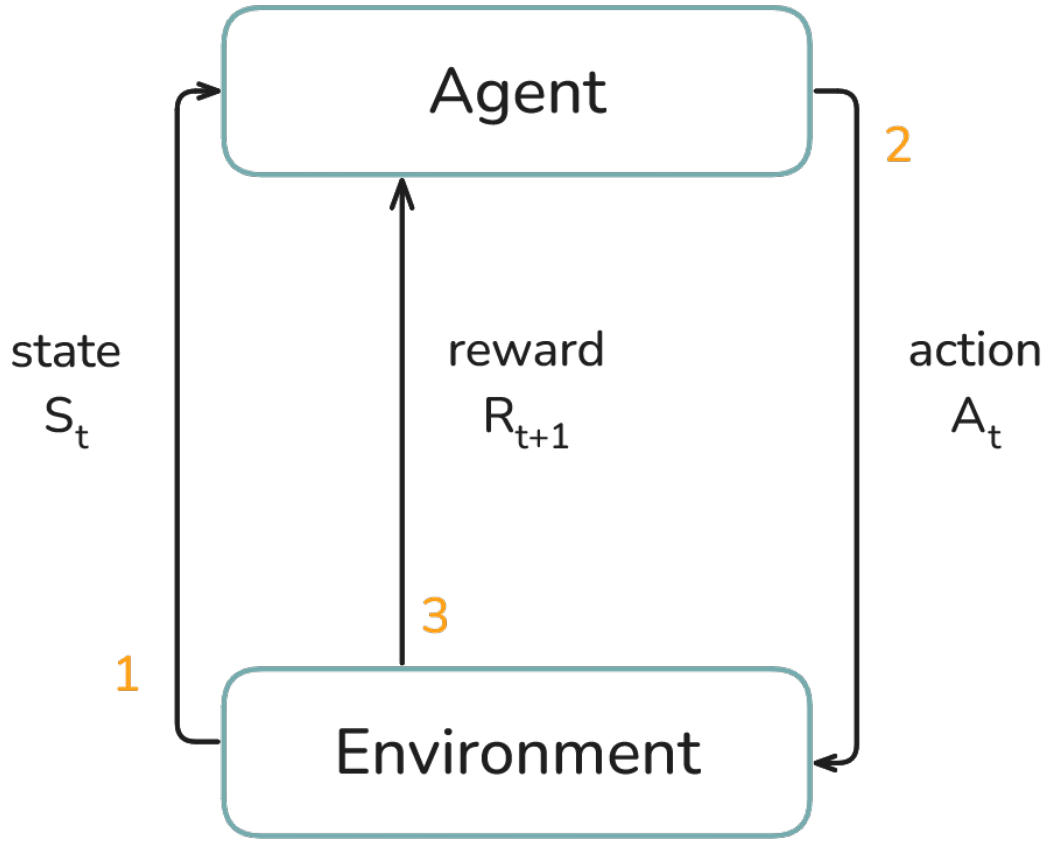


Figure 3.1: At time t, the agent receives the current state S_t. Then the agent picks an action A_t. After that, the environment transitions to a new state S_{t+1} and outputs the corresponding reward R_{t+1}.

Note that this is an example of a fully observable environment. In many problems, the agent sees only an incomplete observation O_t instead of the whole state S_t. In the diagram presented above, these are taken to be equivalent, since S_t is used

instead of O_t.

More specifically, in our work, the observations given to the agent are comprised of: a proficiency dictionary and a target list. The proficiency dictionary reflects how good the agent is at each topic at time t, and the target list has all the input topics which a human might want to improve on. The action is an integer that corresponds with the ID of a LeetCode problem, and the reward is also an integer, with negative values representing "bad" problems and positive values representing "good" problems. More details are to be given in the fifth chapter, second section.

3.2 Elements of Reinforcement Learning

This section follows the framework of [SB18], unless noted otherwise.

The diagram presented above, Fig. 3.1, can be formally defined as a Markov Decision Process (MDP) [SB18].

Definition 3.2.1 (Markov Decision Process). *A **Markov Decision Process (MDP)** is a framework used to model sequential decision-making in fully observable environments. It is an environment where all states have the Markov Property. It is a 5-tuple* $< S, A, P, R, \gamma >$.

- S is a finite set of states;
- A is a finite set of actions;
- P is a state transition probability matrix; $P^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$ is the probability of moving from state s to state s' after taking action a_t in state s_t.
- R is a reward function; $R^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the immediate reward received after transitioning from state t to state $t + 1$.
- γ is a discount factor $\gamma \in [0, 1]$;

Definition 3.2.2 (Markov Property). *The **Markov Property** asserts that the future state s_{t+1} and reward r_{t+1} depend only on the current state s_t and action a_t, not on prior history. Formally:*

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}, r_{t+1} \mid s_t, a_t) \tag{3.1}$$

This "memoryless" property simplifies modeling by reducing the decision problem to only the present state and not the full path from start to time t. Without it, agents would need to track exhaustive histories, making learning not feasible for

complex environments.

The major elements of a RL agent are the following: policy, value function, and model. Each agent can have one or more of these elements.

Definition 3.2.3 (Policy)**.** *A **policy** π is a distribution over actions given states,*

$$\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s] \tag{3.2}$$

The policy is essentially the behavior of an agent. It is stationary or time-independent with respect to t:

$$A_t \sim \pi(\cdot|S_t), \forall t > 0$$

This means that the policy $\pi(a|s)$ does not explicitly depend on the time step t when making a decision.

Definition 3.2.4 (State-value Function)**.** *The **state-value function** $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π:*

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \tag{3.3}$$

Definition 3.2.5 (Action-value Function)**.** *The **action-value function** $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π:*

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{3.4}$$

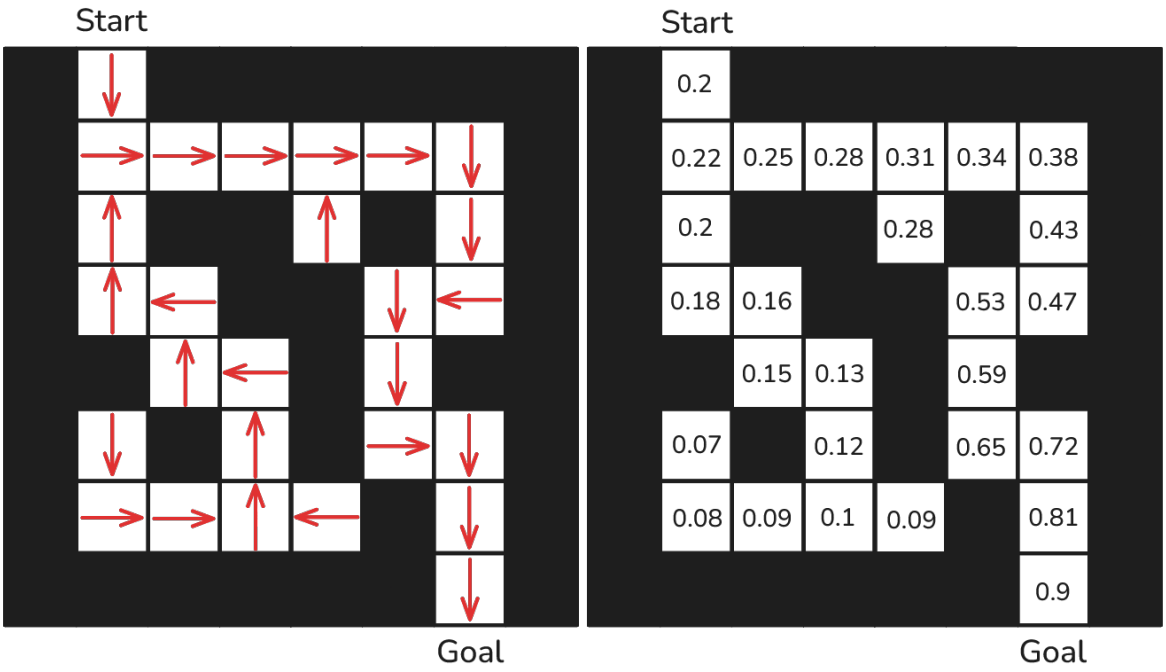


Figure 3.2: Graphical example (reproduced from [Sil15], Lecture 1) of the outputs of a policy, on the left, and a value function, on the right. Note that, in this specific case, the policy appears to be deterministic. A policy is typically stochastic, meaning it assigns a probability distribution over all possible actions for each state.

These state-value and action-value functions can be defined recursively using the Bellman Expectation Equations. Both of these functions are decomposed into the immediate reward plus the discounted value function of the next state.

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right] \tag{3.5}$$

$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right] \tag{3.6}$$

The goal of a reinforcement learning problem is to find the optimal policy. There is always a deterministic optimal policy for any Markov Decision Process (MDP) and it is defined as follows.

Definition 3.2.6 (Optimal Policy)**.** *Given an (MDP), a policy π^* is **optimal** if it maximizes the expected cumulative reward from all states:*

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases} \tag{3.7}$$

where $q_*(s, a)$ is the optimal state-action value function.

In general, MDPs do not admit closed-form solutions for optimal policies or value functions. This implies the necessity of iterative methods like value iteration and policy iteration to approximate solutions. These methods are enabled by the Bellman Equations (Eq. (3.6) and Eq. (3.5)), hence their importance. The iteration of value functions and policies implies: evaluating these functions and then improving upon them.

3.2.1 Evaluating Values and Policies

Temporal Difference is one of the most common algorithms for the evaluation of an action-value function. Temporal Difference (TD) is a method that updates estimates incrementally by predicting future values based on current estimates. TD learning is a combination of Monte Carlo methods and dynamic programming. These algorithms do not need to wait until the end of an episode to update value estimates and do not require a complete model of the environment, unlike Monte Carlo and dynamic programming solutions. This makes TD learning particularly efficient for online learning.

The simplest form, TD(0), updates the value function as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right] \tag{3.8}$$

where:

- α is the learning rate, controlling the update step size;
- γ is the discount factor $\gamma \in [0, 1]$;
- $R_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$ is called the **TD target** and it represents a prediction of $Q(s_t, a_t)$;
- $R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$ is the **TD error**, which measures the difference between the predicted and observed outcomes;

Realistically, the state-action space of environments can get so large that storing these values is not feasible. Because of this, value function approximation is used.

Value function approximation is a technique used to predict the value function's output. Instead of storing exact values for every state or state-action pair in a tabular way, approximation methods are used to generalize from data. This is done by using parameterized functions like linear models or neural networks.

3.2.2 Improving Values and Policies

For value function approximation, improving the predictions can be done using gradient descent. In this case, the objective function from which the gradient is taken is the TD error, with the goal being to minimize it. We can use gradient descent as follows:

$$\Delta w = \alpha \left(R_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}, w) - \hat{V}(s_t, w) \right) \nabla_w \hat{Q}(s_t, a_t, w) \tag{3.9}$$

where:

- α is the step size or learning rate;
- $\hat{Q}(s_{t+1}, a_{t+1}, w)$ is the guess made by the model for state s_t and action a_t with weights w;
- $\nabla_w \hat{Q}(s_t, a_t, w)$ is the gradient of the action-value function with weights w;

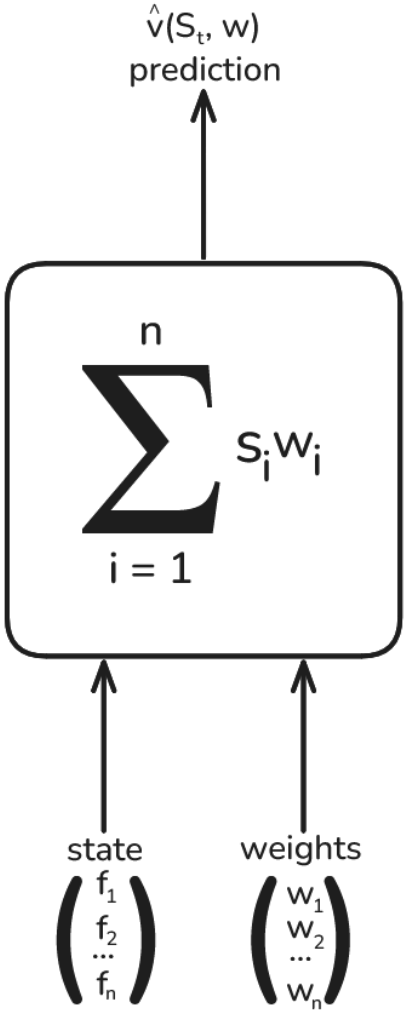


Figure 3.3: Schematic of a simple linear function model. Observe that f_n are features of each state and w_n are weights corresponding to each feature.

The policy gradient method [SMSM00] uses gradient ascent with respect to another, different, objective function. This new objective function is, most commonly, the cumulative reward. The parameters are being updated in the same way as previously seen:

$$\Delta\theta = \alpha \left(R_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}, w) - \hat{Q}(s_t, a_t, w) \right) \nabla_\theta \log \pi_\theta(s, a) \quad (3.10)$$

where:

- α is the step size or learning rate;
- $\hat{Q}(s_{t+1}, a_{t+1}, w)$ is the guess made by the model for state s_{t+1} and action a_{t+1} with weights w;
- $\nabla_\theta \log \pi_\theta(s, a)$ is the gradient of the policy with weights θ;

Having laid the groundwork for Reinforcement Learning through these concepts, we now turn our focus to the algorithm proposed, Advantage Actor-Critic.

3.3 Advantage Actor-Critic Method

Advantage Actor-Critic (A2C) [MBM+16] is a synchronous, on-policy reinforcement learning algorithm that combines policy gradient methods with value function approximation. It features an actor and a critic. The actor parameterizes the policy and selects actions, while the critic parameterizes the action-state function and estimates it.

3.3.1 Motivations

The Advantage Actor-Critic (A2C) algorithm was introduced by OpenAI in 2016 [Ope17] as a synchronous, simplified variant of their earlier Asynchronous Advantage Actor-Critic (A3C) method. The motivation behind A2C was to keep the benefits of policy gradient with low-variance advantage estimates, but avoid the complexity of asynchronous updates. OpenAI demonstrated that A2C could achieve comparable performance to A3C in many environments while being easier to implement and tune. This made it popular in modern deep reinforcement learning.

3.3.2 Formal Presentation

Before presenting the pseudocode of the algorithm, there are a few concepts that need to be explained. This Advantage Actor-Critic algorithm uses Generalized Ad-

vantage Estimations (GAE) [SML+15] instead of raw state-action values.

Advantage estimates quantify how much better a specific action is compared to the average action in a state, and it is defined as $A(s,a) = Q(s,a) - V(s)$. The TD error is an unbiased estimate of this advantage. This can be derived from the Bellman equation, where $\mathbb{E}[r_t + \gamma V(s_{t+1})] = Q(s_t, a_t)$. Advantage estimates are very important in actor-critic methods because they reduce variance by subtracting a baseline ($V(s)$), therefore stabilizing the updates.

To further improve this estimation, Generalized Advantage Estimation (GAE) combines multi-step TD errors using an exponential weighting scheme controlled by $\lambda \in [0, 1]$. The GAE formula is:

$$A_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{3.11}$$

where:

- δ_{t+l} is the TD-error at time $t + l$;
- γ is the discount factor;
- λ is a constant that adjusts the bias-variance trade-off since lower λ is similar to TD-style bootstrapping, while $\lambda \approx 1$ leans is comparable to full Monte Carlo returns;

Below we present the general algorithm, without specific tuning, of OpenAI and Stable-Baselines3's [RHG+21] advantage actor-critic method:

Algorithm 1 Advantage Actor-Critic with GAE [RHG+21]

function A2C(π_θ, V_ϕ, n_steps, γ, λ, $c1$, $c2$, $c3$)
  Initialize policy parameters θ and value function parameters ϕ
  while not converged **do**
    Initialize empty buffers: S, A, R, D, V
    $s_0 \leftarrow$ initial state

    for $t = 0$ to $n_steps - 1$ **do**
      Sample action $a_t \sim \pi_\theta(a_t|s_t)$
      Take action a_t, observe r_t, s_{t+1}, d_t
      Store s_t, a_t, r_t, d_t
      $V_t \leftarrow V_\phi(s_t)$, store in V
      if d_t **then**
        $s_{t+1} \leftarrow$ initial state
      end if
      $s_t \leftarrow s_{t+1}$
    end for

    Compute $V_n \leftarrow V_\phi(s_n)$
    Initialize $\hat{A}_t \leftarrow 0$, $A \leftarrow$ empty list
    for $t = n_steps - 1$ to 0 **do**
      $\delta_t \leftarrow r_t + \gamma(1 - d_t)V_{t+1} - V_t$
      $\hat{A}_t \leftarrow \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$
      Prepend $\hat{A}_t$ to A
    end for

    for $t = 0$ to $n_steps - 1$ **do**
      $\hat{R}_t \leftarrow \hat{A}_t + V_t$
    end for

    $\mathcal{L}_{\text{policy}} \leftarrow -\frac{1}{n_steps} \sum_t \hat{A}_t \log \pi_\theta(a_t|s_t)$
    $\mathcal{L}_{\text{value}} \leftarrow \frac{1}{n_steps} \sum_t (\hat{R}_t - V_\phi(s_t))^2$
    $\mathcal{L}_{\text{entropy}} \leftarrow -\frac{1}{n_steps} \sum_t \mathcal{H}[\pi_\theta(\cdot|s_t)]$
    $\mathcal{L}_{\text{total}} \leftarrow c_1 \cdot \mathcal{L}_{\text{policy}} + c_2 \cdot \mathcal{L}_{\text{value}} + c_3 \cdot \mathcal{L}_{\text{entropy}}$

    Update θ, ϕ using gradient descent on $\mathcal{L}_{\text{total}}$
  end while
end function

Chapter 4

Literature survey

This following chapter explores existing work on reinforcement learning for recommender systems.

One of the most popular challenges with RL systems for recommendation is the need for online training, that is, to interact with real users in real time. Since RL approaches learn very slowly and are sample inefficient, users will suffer through many bad recommendations before seeing a good one [ZZX+19]. In consequence of this, all of the next presented papers use combinations of: simulating training using offline data (logs or histories from users) and online training. All the approaches presented next work on improving offline training. Whether that is by creating an accurate and dynamic simulation of users' interactions [ZZX+19, ZXD+20] or recommending lists of actions instead of just one, or using Knowledge Graphs (KG) [ZDC+20] for rich state embeddings.

4.1 Deep-Q-Learning Methods

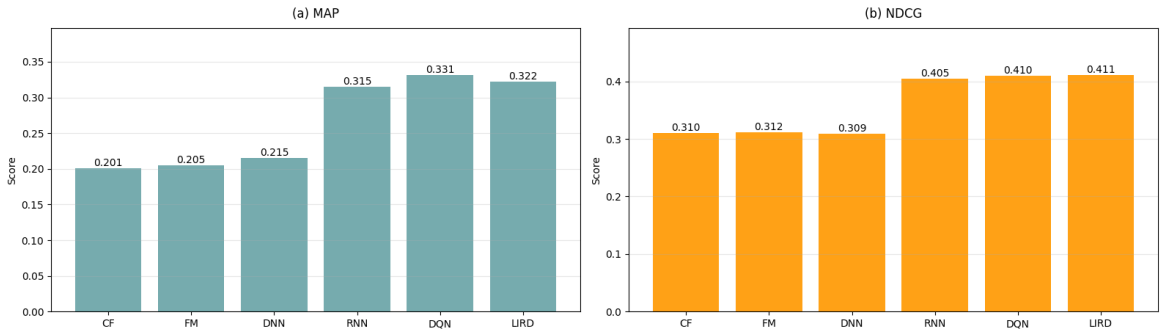
The work presented in the paper "Deep Reinforcement Learning for List-wise Recommendations" [ZZX+19] is one of the first instances of a Recommender Agent (RA) being modeled as an MDP and solved using Deep-Q-Learning (DQN). Xiangyu Zhao et al. work to solve the limitations of traditional recommenders. Those are: the short-term bias and the static recommendation process that supervised approaches suffer from. The proposed approach, named LIRD, maximizes the cumulative rewards over a session rather than just immediate clicks or purchases.

To use offline training from user history, LIRD uses an online environment simulator. Each entry in the memory records a triple $((state, action) \rightarrow reward)$, where the state is the user's N most recently clicked items, and the action is a list of K recommended items. When simulating a new recommendation, LIRD computes the cosine similarity between the current $(state, action)$ pair and stored histories and

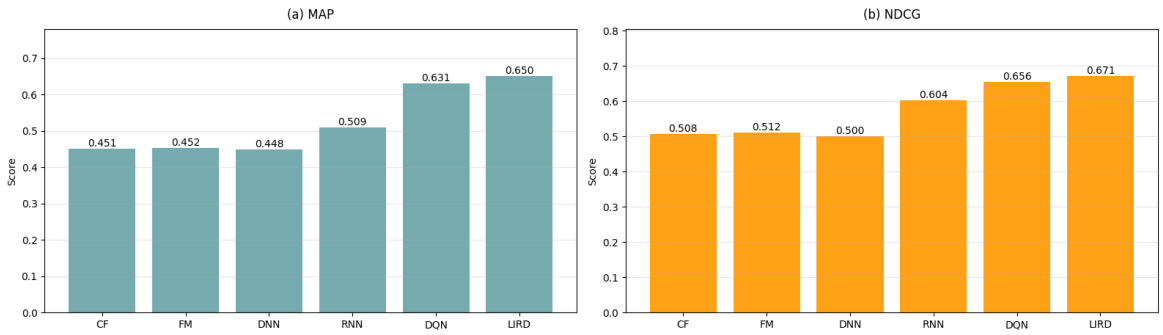
stochastically samples a reward. This approach overcomes the variance problem. That is, even if there is no sample of some exact $(state, action)$ pair, a moderately accurate reward is still being given.

LIRD uses an Actor-Critic architecture for its online simulator. The actor network maps the current state to K distinct weight vectors, each defining a Neural Network (NN). The actor then selects the best action and removes it from the candidate pool. The Critic network is a DQN approximator, which estimates the expected Q-values. Unlike classical DQN that evaluates all actions, LIRD's actor output allows the critic to predict the Q-values of only the K actions picked by the actor.

LIRD was trained on a real-world e-commerce dataset with over 1 million items and was benchmarked against common baselines: Collaborative Filtering (CF), Factorization Machines (FM), Deep Neural Nets (DNN), Recurrent Neural Networks (RNN), and standard DQN. LIRD outperformed all these baselines in both the short-term and long-term sessions. Notably, only the two RL approaches (standard DQN and LIRD) maintained gains in longer sessions.



(a) Short-term session



(b) Long-term session

Figure 4.1: Comparison (reproduced from [ZZX+19]) between short and long term sessions, using MAP [TS06] NDCG [JK02] metrics.

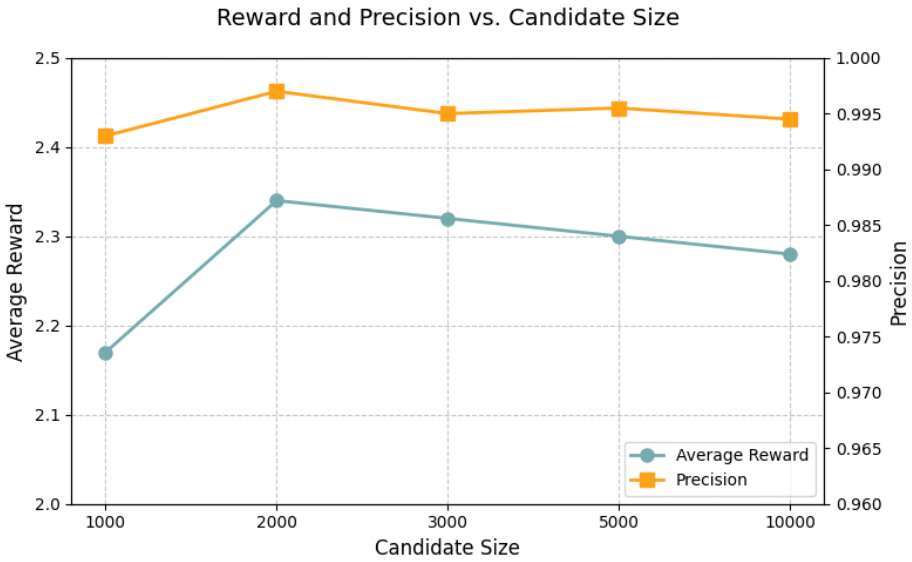
The second Deep-Q-Learning system, presented in "Interactive Recommender System via Knowledge Graph-enhanced Reinforcement Learning" [ZDC+20] by Sijin Zhou et al. addresses two problems with sequential recommendation: RL's low

sample efficiency and noise (needing a ton of interaction with the data before achieving relevant results) and the enormous action space of possible items. To overcome this, the authors propose guiding the learning process with a Knowledge Graph (KG). By adding semantic relationships to item and candidate representations, this system learns faster and recommends more relevant items.

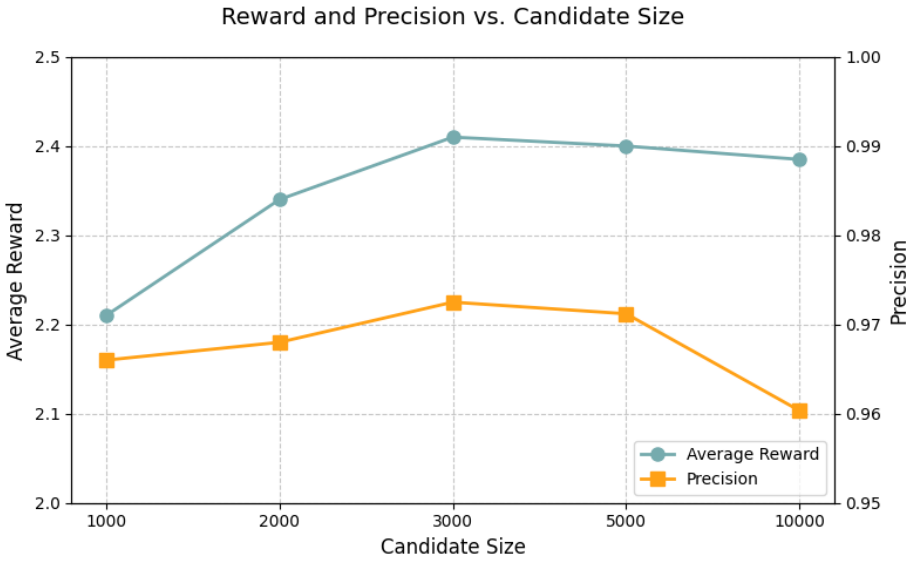
This method, named KGQR, uses three components. A Graph Convolutional Network (GCN) creates accurate embeddings for each item by aggregating information from its k-hop neighbors in the KG. A Gated Recurrent Unit (GRU) models the user's previous interactions by taking as input these embeddings and outputting a 64-dimensional vector representing a user's current state. Each individual item embedding is concatenated with the user's state to create the states that the RL algorithm works with. These are used in a dueling double-DQN that estimates Q-values over these states or observations. The top item is selected under a $\epsilon - greedy$ policy.

The main method that addresses these problems is the candidate selection strategy. Instead of scoring all items, which would not be feasible, the system gathers the k-hop KG neighborhood around the user's positive clicks and forms a much smaller candidate pool because of this.

Experiments were conducted on 2 datasets, namely Book-Crossing [ZMKL05] and MovieLens [HK15], and show that KGQR consistently outperforms several baselines (multi-arm bandits, standard DQN). KGQR reaches 3–5% higher cumulative reward and moderate gains in precision and recall while using fewer than 20% of the interactions required by normal DQN. The authors also identify the optimal candidate pool size ranges.



(a) Book-Crossing Dataset



(b) Movie-Lens Dataset

Figure 4.2: Comparison (reproduced from [ZDC+20]) of reward and precision between the two datasets.

4.2 Model-Based Methods

The paper "Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation" [ZXD+20] by Lixin Zou et al. addresses the challenge of training reinforcement-learning–based recommenders from offline logs. Temporal Difference Learning suffers from divergence when combining function approximation, bootstrapping, and offline training. Also, Monte Carlo Methods are very computationally inefficient and need to be trained online.

The proposed model, Pseudo Dyna-Q (PDQ) uses two main components. A Q-network for learning a policy and a dynamic simulator, named the world model. This world model is a network that takes as input a state–item pair and predicts both the user's feedback (click, skip, purchase) and the probability of session termination. Every time-step, the world model is updated using importance-sampling–corrected logged trajectories. Since those logs were created by an older policy, each (s, i) pair is weighted by how likely the new policy would have chosen that same pair. These weights are also capped, so there aren't any examples that dominate throughout many policies.

Next, PDQ trains its Q-network through simulated interactions with this world model. At each simulated step, the agent selects an item via an $\epsilon - greedy$ policy, receives a "simulated reward" from the world model, and observes the next state. These simulated transitions are stored in a replay buffer together with real logged transitions. After this, the Q-network is updated by minimizing the mean squared error using both real and simulated data.

PDQ was trained on the Taobao ($\sim$100 M events) [Gro18] and Retailrocket ($\sim$1 M events) [Ret16] datasets and benchmarked against off-policy RL algorithms (DQN-R and DDPG-KNN), RNN-based methods (GRU4Rec) and other popular recommenders (factorization machines). The authors also benchmarked a naive PDQ implementation (called PDQ-N), in which the world model is only trained at the start. PDQ's full variant has the highest average clicks per session, outperforming almost all baselines with statistical significance. Additionally, ablation tests prove that importance clipping alone provides significant gains, as shown in the table below.

Table 4.1: Performance on Taobao and Retailrocket (Epoch = 20 000)

Agent	Taobao		Retailrocket	
	Clicks	Diversity	Clicks	Diversity
FPMC	1.8394	0.0441	1.8127	0.0405
GRU4Rec	1.8421	0.0435	1.9272	0.0831
DQN-R	0.8810	0.0006	0.6040	0.0005
DDPG-KNN (k=1)	0.8678	0.0004	0.5957	0.0008
DDPG-KNN (k=0.1 N)	0.8745	0.0007	0.6050	0.0017
DDPG-KNN (k=N)	0.8624	0.0157	0.8942	0.0008
PDQ (N)	1.7830	0.2491	2.2471	0.1373
PDQ	2.7998*	0.4182*	2.8131*	0.1988*

An asterisk (*) indicates a statistically significant improvement ($p < 0.01$) over the best baseline.

Chapter 5

a2cl: Web Application for Tracking Progress

In this chapter, we present the development of a2cl [Lau25], a web application serving as a proof-of-concept for the underlying Actor-Critic model. In the first section, the full iterative process of the development of the model will be presented. After that, the second section details the application's architecture and data flow.

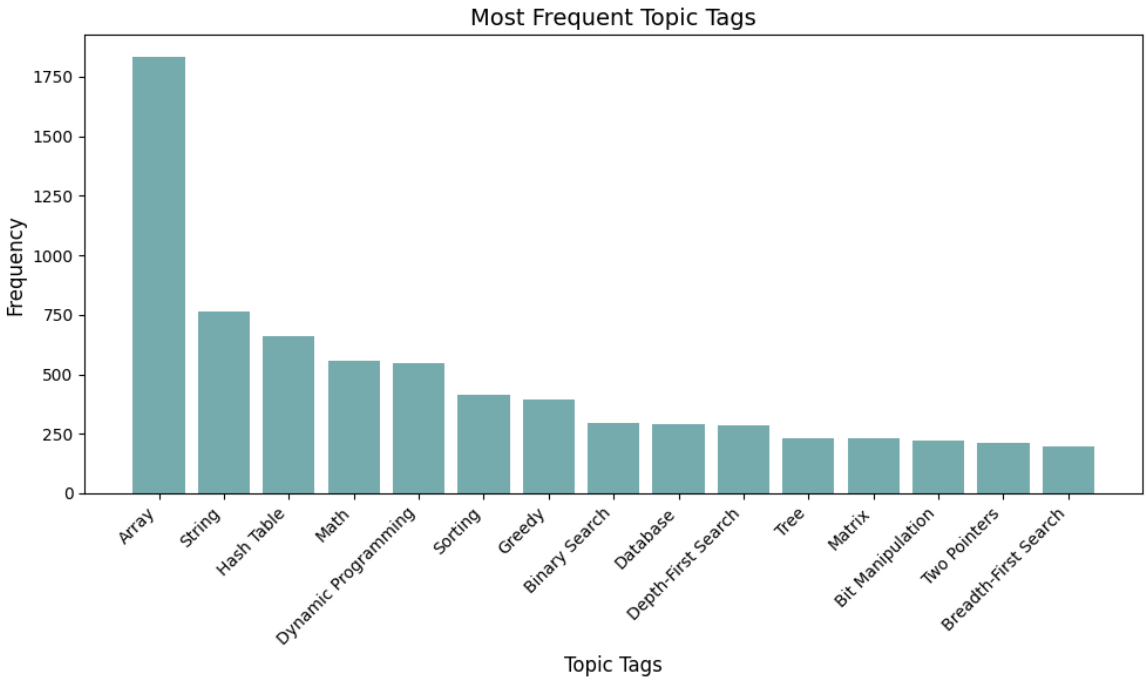
5.1 Model Development

The development of our proposed Actor-Critic method involves analyzing the available questions on LeetCode in order to efficiently define the problem. Each topic from each question will have assigned a difficulty score (from 0 to 100) by leveraging statistical information gathered through the previously mentioned data analysis. After that, the main elements of our training environment will be presented. We then perform hyperparameter tuning in order to optimize the model's performance. Finally, we provide the final results.

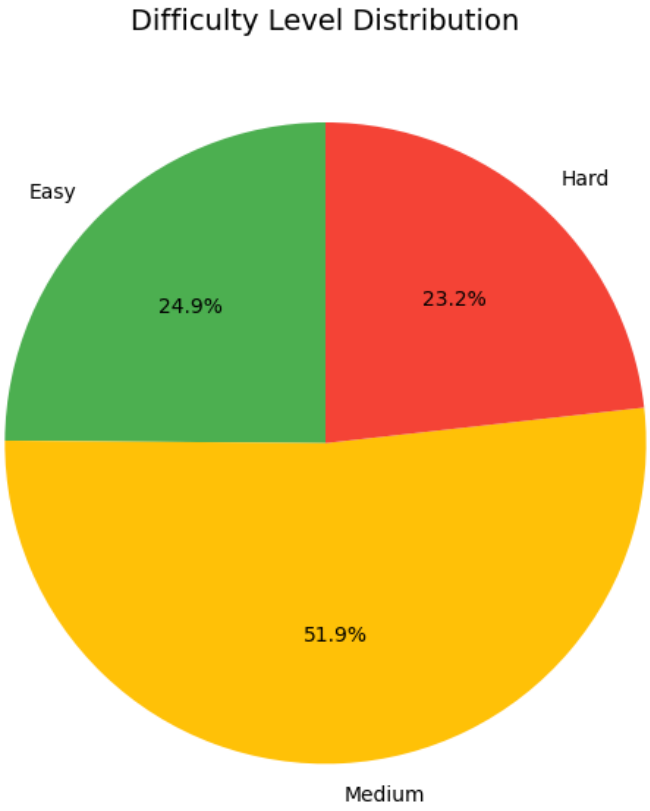
5.1.1 Data Analysis

This subsection serves 2 primary goals. Firstly, we explore the LeetCode problems dataset for potential patterns, biases, or anomalies that can heavily impact the final model. Also, gathering some statistical information (mean, median, variance, lower and upper acceptance rate bounds) that will be used in the next chapter for grading each problem.

We begin with an examination of topic frequency to understand how problems are distributed across different categories. Topics dominating the dataset will introduce bias in model training. For the same reason, we will look at the distribution of difficulties.



(a) Most frequent 15 topic tags



(b) The distribution of difficulties

Figure 5.1: Topics and difficulty distribution analysis. Notice that the topic set is very unbalanced, with only the "Array" topic appearing in roughly half of all problems.

Next, we analyze acceptance rates to detect anomalies or extreme values. An additional reason is that the strategy presented in the next subsection, for grading each topic from a question, assumes a normal distribution of acceptance rates.

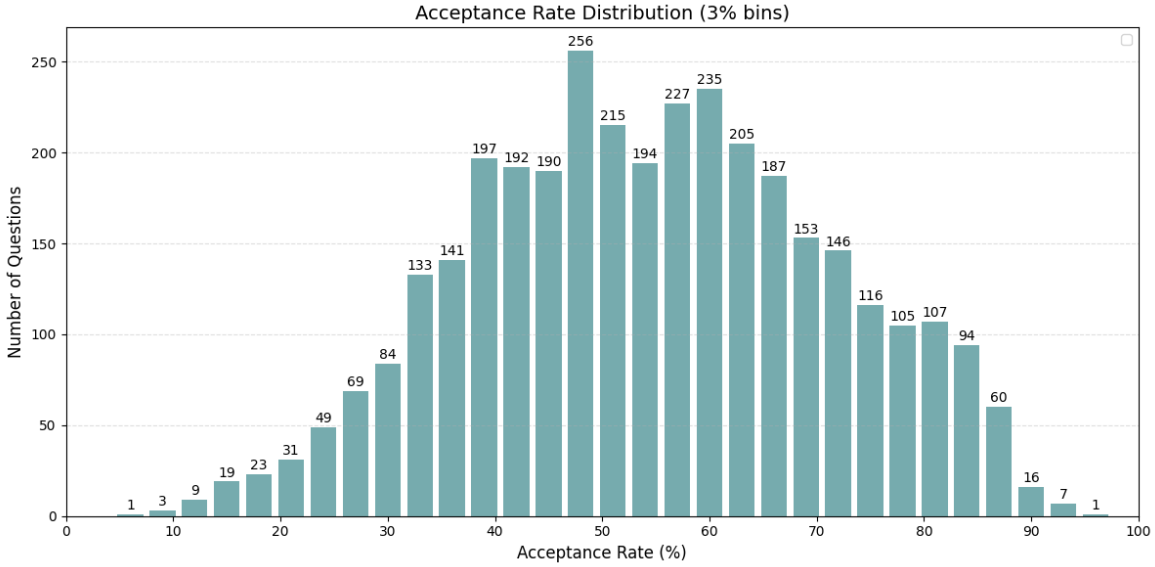


Figure 5.2: Distribution of acceptance rates using 3% bins. Observe that this distribution is in fact normal. Consequently, there tends to be a ”typical” problem acceptance rate which clusters around the center.

After that, we determined the most appropriate measure to use for an average difficulty problem. This was done by checking if there is a significant difference between median and mean acceptance rate values. There were some observed topics with significant discrepancies, which suggest skewed distributions. Consequently, the median was chosen over the mean.

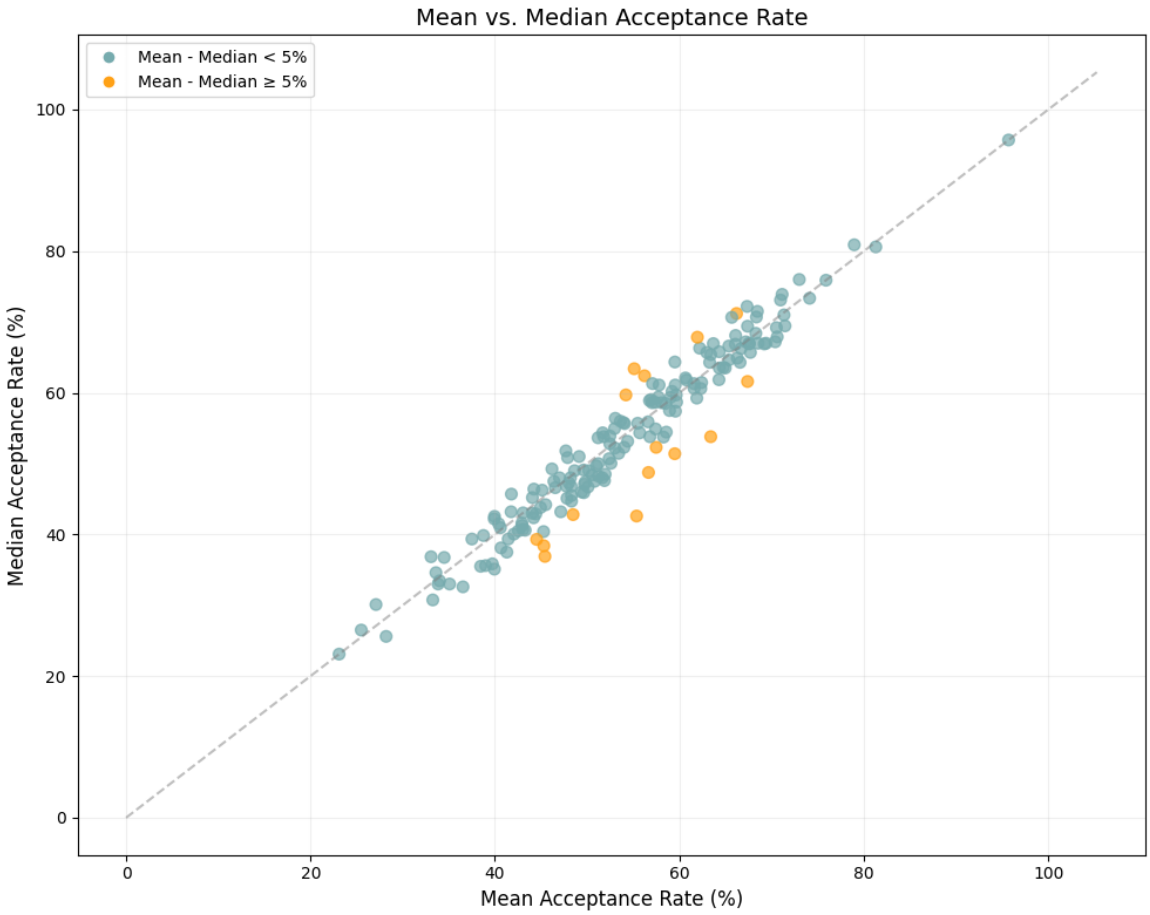


Figure 5.3: The difference between the mean and median acceptance rates.

5.1.2 Data Collection and Pre-Processing

The first phase of data pre-processing was cleaning the raw dataset. All missing questions were manually added. Additionally, many questions with multiple topics had mismatched or incomplete labels. These were all manually corrected as well. After this, acceptance rates were rounded to two decimal places, and difficulty levels (easy, medium, hard) were converted into integers (0, 1, 2).

After this, each topic was given a grade (from 0 to 100) which is meant to resemble difficulty. This grade was based both on problem-level statistics (mean or median, variance, maximum, and minimum acceptance rates per topic) and previously created heuristics. These essentially serve as new features, and the whole process can be categorized as a type of feature engineering.

More specifically, the grading was computed by using the distribution of each $(topic, difficulty)$ pair. Three values from the distribution are used:

- min: the minimum acceptance rate observed
- max: the maximum acceptance rate observed

- median: the median

New ”difficulty values” are overlapped on top of these true values. The ”difficulty values” will be called:

- low: for a low difficulty problem
- typical: for a typical difficulty problem
- high: for a high difficulty problem

Using these new ranges, the distribution was split in $low - high$ an amount of areas. Each area represents one ”difficulty value area”. Grading a specific topic does not require actually computing the distribution and plotting the question. The only information needed is these 6 previously presented points and the width of each area. Crucially, all areas on one side of the median line have equal width. This width can be found using the equations below.

$$LeftSideAreaWidth = \frac{\sigma - \text{Min}}{\text{Typical} - \text{Low}} \tag{5.1}$$

$$RightSideAreaWidth = \frac{\text{Max} - \sigma}{\text{High} - \text{Typical}} \tag{5.2}$$

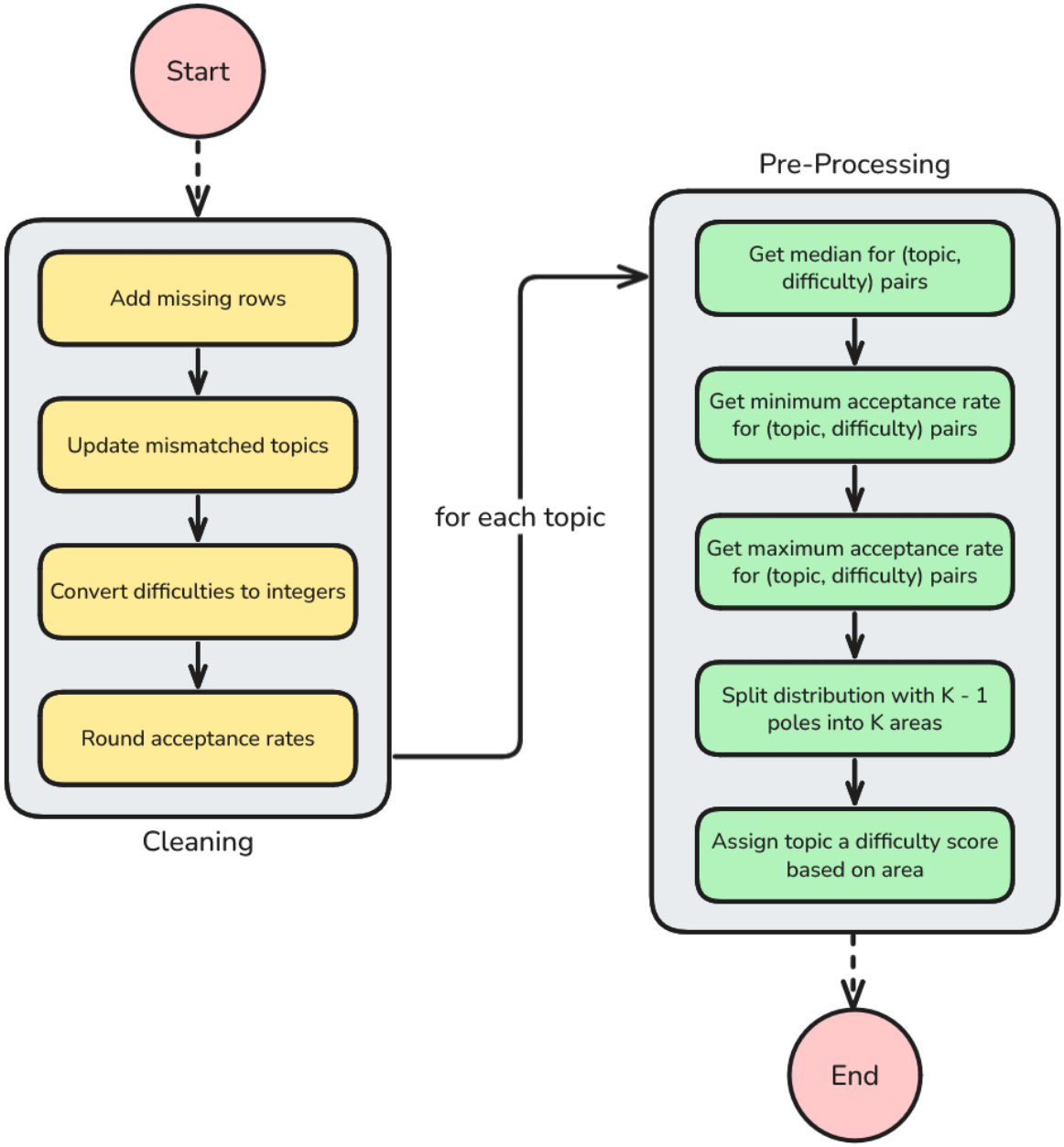


Figure 5.4: Data Cleaning and Pre-Processing pipeline.

These statistics were compiled into a CSV file for reference. The grading algorithm was then applied to assign difficulty scores to each topic, with the result being placed in a JSON file. A summary of the raw data files and their structure is presented below, along with the grading algorithm.

Table 5.1: Summary of the raw data and use.

No.	File Name	Use
1	leetcode_questions_initial.csv	Contains initially scraped raw data
2	leetcode_questions.csv	Pre-processed data, as explained above
3	leetcode_questions_distribution_data.csv	Full distribution data for each $(topic, difficulty)$ pair
4	leetcode_problems_grading.json	Each problem's topics graded by the algorithm

Algorithm 2 Grading Algorithm

```
1:  function GRADE_QUESTIONS(questions, question_distribution, output)
2:      Initialize questions, question_distribution
3:      Initialize graded_questions dictionary

4:      for question in questions do
5:          for topic in question_topics do
6:              Initialize median ← question_distribution

7:              if question_acceptance_rate > median then
8:                  Initialize maximum ← question_distribution
9:                  Initialize grade ← question_distribution
10:                 Initialize right_width ← question_distribution
11:                 while maximum > question_acceptance_rate do
12:                     maximum = maximum − right_width
13:                     grade = grade + 1
14:                 end while
15:             else
16:                 Initialize minimum ← question_distribution
17:                 Initialize grade ← question_distribution
18:                 Initialize left_width ← question_distribution
19:                 while minimum < question_acceptance_rate do
20:                     minimum = minimum + left_width
21:                     grade = grade − 1
22:                 end while
23:             end if

24:             break
25:         end for
26:         Write results to output
27:     end for
28: end function
```

5.1.3 Development of the Environment

The Leetcode recommender environment, created using the Gymnasium library [Far22], provides a controlled setting where the A2C model can take actions (equivalent to solving a problem) and learn patterns. In this subsection, we present the critical decisions made in developing the environment, which span 5 important aspects.

State and Action Representation. Actions are discrete integers that, for backward compatibility reasons, represent the real IDs of LeetCode problems. States are made to encompass two important features: the current proficiency of the agent and the target topics. Proficiencies are represented as an array of `length() == number_of_topics`. Each element in the array has a discrete value from 0 (complete beginner) to 100 (master). The target topics are represented as a one-hot encoded array. A value of 1 indicates the presence of that specific topic in the input, while 0 denotes its absence. Both these features assure full observability of the environment and are wrapped in a dictionary and form the complete state.

Reward Scheme. Multiple reward functions were designed to balance immediate feedback and long-term objectives. The best performing scheme combines sparse rewards for solving a problem matching current proficiencies with dense rewards that award healthy proficiency progression. Additionally, penalties are applied for repeated or too difficult problems solved.

$$\text{reward} = \begin{cases} -2 & \text{if a repeated problem is solved} \\ +3 & \text{if a problem with the correct problem is solved} \\ [+1, +3] & \text{if a problem with appropriate difficulty is solved} \\ -1 & \text{if a problem with inappropriate difficulty is solved} \end{cases} \tag{5.3}$$

To encourage consistent incremental progress, the reward function uses a shaped component proportional to topic completion. This bonus reward is calculated as:

$$r_{\text{progress}} = \eta \cdot \frac{p_t}{100} \tag{5.4}$$

where:

- p_t is the current proficiency value for topic t (range: 0-100)
- η is a scaling factor controlling the importance of partial progress

State Transition Logic. Transitions between states follow deterministic rules based only on the current state and selected action. These transitions give a dy-

namic property to the algorithm and allow new, more difficult, problems to enter the ”solvable” range. The transitions are proportional to the reward given.

Episode Termination Conditions. An episode terminates when either of two conditions is met: the agent reaches or exceeds a defined proficiency threshold (done), or a maximum step count is exceeded (truncated).

Environment Reset Protocol. The reset operation returns the environment to a valid initial state. It has polymorphic behavior based on the instance of the environment. If used with a training environment, the target topics will shift randomly. If used with a testing environment, the same target topics will be preserved. Independently of these two, the array tracking the previous problem sequence will be emptied and correct default proficiencies re-initialized.

The environment has 2 main methods:

- `step()`: is responsible for computing and applying the dynamics of the environment one time step at a time. More specifically, it transitions to the new state, applies the reward, stores previously solved problems, and returns this information.
- `reset()`: resets the environment to its initial state and returns that initial observation.

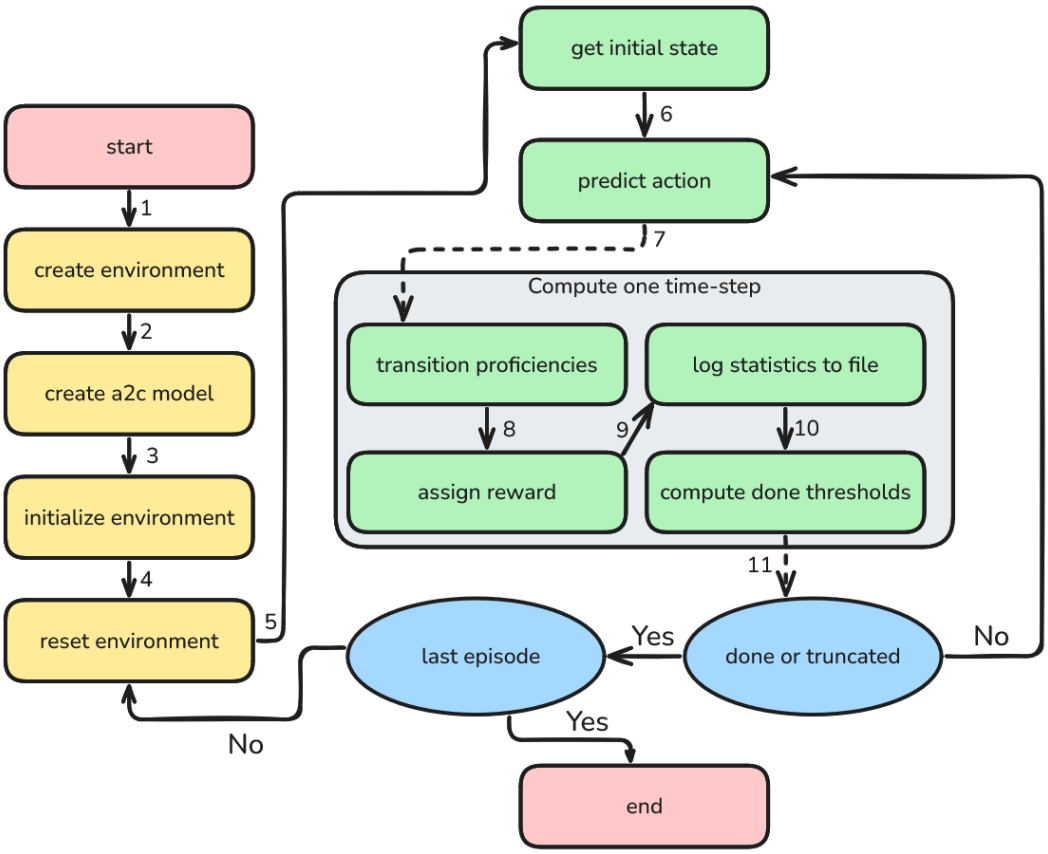


Figure 5.5: Diagram of the LeetCode recommender environment flow.

5.1.4 Hyperparameter Tuning

The process of empirically finding the best hyperparameters was split into two phases. In the first phase, we compared 6 different reward schemes and chose the most promising one. After this, we iterated over many combinations of network-specific parameters and empirically selected the best one. Below we present the different schemes. Each value is the corresponding reward gain or loss for a specific event. These are, in order: `(Repeated Action, Threshold Exceeded, Correct Topic Selected, Suboptimal Problem Selected, Optimal Problem Selected, Basic Difficulty Problem Selected, Advanced Difficulty Problem Selected, Overly Simple Problem Selected, Excessively Challenging Problem Selected)`

Table 5.2: Summary of the different reward schemes chosen

Scheme	Values
aggressive	(-20, +100, +20, +3, +2, +1, +1, +0, -1)
moderate	(-10, +100, +10, +3, +2, +1, +1, +0, -1)
mild	(-5, +100, +5, +3, +2, +1, +1, +0, -1)
repeated action over right topic	(-20, +100, +10, +3, +2, +1, +1, +0, -1)
right **t**opic **o**ver **r**epeated **a**ction	(-10, +100, +20, +3, +2, +1, +1, +0, -1)
RTORA + dense rewards	(-2, +100, +10, +2, +3, +1, +1, +0, -1, +sum)

We then started the experiments, using the same model parameters, tracking the moving average reward from the past 100 episodes. After empirically testing the schemes, for 1 million time-steps, the scheme augmented with denser rewards was chosen for advancing to the second phase. Below are the results of the first stage. Note that other small tweaks were also tested from each scheme, but we show only the main schemes themselves.

Table 5.3: Summary of results from the first phase

Scheme	Moving Average Reward
aggressive	-362
moderate	-259
mild	-117
repeated action over right topic	-382
right topic over repeated action (RTORA)	-105
RTORA + dense rewards	398

During the second stage, initial testing focused on higher-level parameters (network structure, actor and critic loss coefficients, and orthogonal initialization set-

tings). These elements significantly influence model behavior. After this, we fine-tuned the more granular training parameters (learning rate, entropy coefficient, and gradient clipping thresholds). All of these secondary parameters operate within constraints created by the higher-level parameters. Next, we detail the final optimal configuration that resulted from the previously mentioned approach and explain each parameter and the values under experiments.

```
A2C_CONFIG = {
    ent_coef=0.002,
    vf_coef=0.2,
    n_steps=32,
    gae_lambda=0.95,
    max_grad_norm=0.5,
    policy_kwargs={
        net_arch={
            pi: [64, 64, 32],
            vf: [64, 64]
        },
        optimizer_class=torch.optim.Adam,
        optimizer_kwargs={
            weight_decay: 0.0,
            eps: 1e-6
        },
    },
    use_rms_prop=False,
    gamma=0.7
}
```

1. `ent_coef` (0.002), `default=0`: The entropy coefficient penalizes low-entropy policies, therefore increasing exploration. A small positive value successfully balances exploration and exploitation.

$$\text{ent_coef} \in \{0.3, 0.2, 0.1, 0.007, 0.005, 0.002, 0.001, 0\} \tag{5.5}$$

2. `vf_coef` (0.2), `default=0.5`: Scales the value function loss relative to the policy gradient loss. A smaller ratio stops critic domination while still providing sufficient value prediction accuracy.

$$\text{vf_coef} \in \{0.1, 0.2, 0.3, 0.4, 0.5\} \tag{5.6}$$

3. `n_steps` (32), `default=5`: Defines the rollout length before parameter updates. 32 steps lower variance significantly while keeping learning fast.

$$\text{n_steps} \in \{5, 16, 32, 64, 128\} \tag{5.7}$$

5. `gae_lambda` (0.95), `default=1`: Generalized Advantage Estimation parameter that controls the bias-variance tradeoff. 0.95 emphasizes recent rewards but also maintains moderate variance reduction.

$$\text{gae_lambda} \in \{0.95, 0.9\} \tag{5.8}$$

6. `max_grad_norm` (0.5), `default=0.5`: Gradient clipping threshold made for preventing exploding gradients. This limit allows meaningful updates and stabilizes training.

$$\text{max_grad_norm} \in \{0.5, 0.4\} \tag{5.9}$$

7. `pi_architecture` ([64, 64, 32]), `default=[64, 64]`: Defines the complexity of the actor's neural network. Facing actor underfitting, the updated network captures the objective function more efficiently.

$$\text{pi_architecture} \in \{[64, 64, 32], [128, 128], [256, 256]\} \tag{5.10}$$

8. `vf_architecture` ([64, 64]), `default=[64, 64]`: Defines the complexity of the critic's neural network. Experiments did not reveal any significant overfitting or underfitting.

$$\text{vf_architecture} \in \{[64, 64, 32], [64, 64, 64]\} \tag{5.11}$$

9. `optimizer_class` (Adam), `default=RMSProp`: Explicitly selects Adam as the optimizer. Experiments showed benefits when using sparse reward signals.

$$\text{optimizer_class} \in \{Adam, RMSProp\} \tag{5.12}$$

11. `gamma` (0.7), `default=0.99`: The discount factor that determines present value of future rewards. 0.7 emphasizes near-term rewards in this environment.

$$\text{gamma} \in \{0.99, 0.9, 0.8, 0.7\} \tag{5.13}$$

5.1.5 Results

The final scores, using our most efficient model, are presented below.

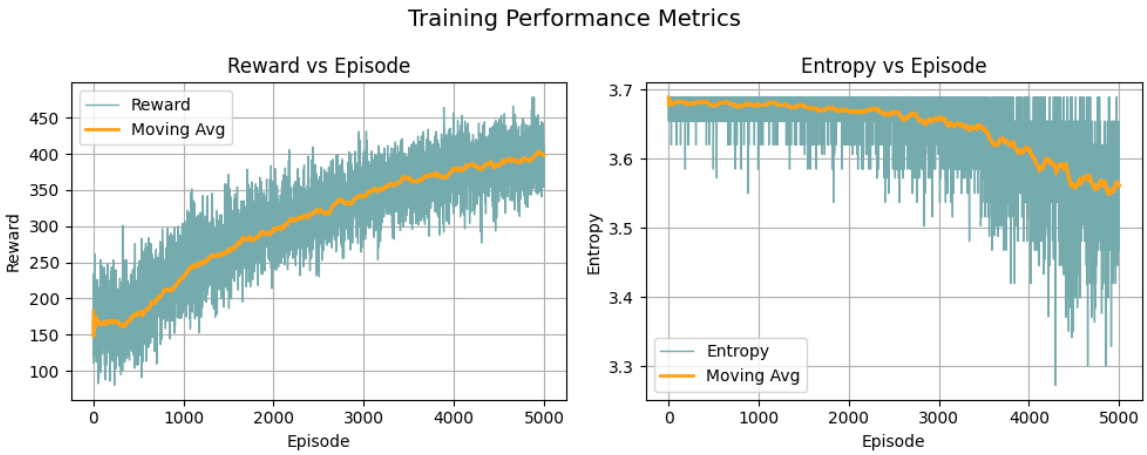


Figure 5.6: Reward and Entropy per Episode results after 5000 episodes.

The reward progression demonstrates steady improvement. The moving average shows this trend clearly, smoothing out the very high variance characteristics of this algorithm. Entropy metrics show a corresponding pattern, high initial values prove healthy exploration, followed by a gradual decline as the policy becomes more deterministic. This entropy curve suggests an appropriate exploration-exploitation balance in the beginning, although the final entropy levels do not get to converge low enough.

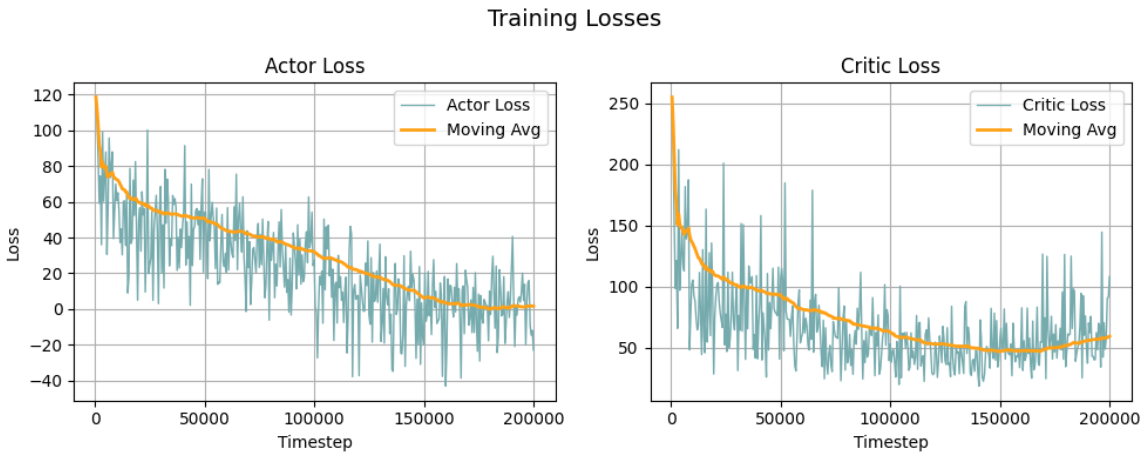


Figure 5.7: Actor and Critic losses during training.

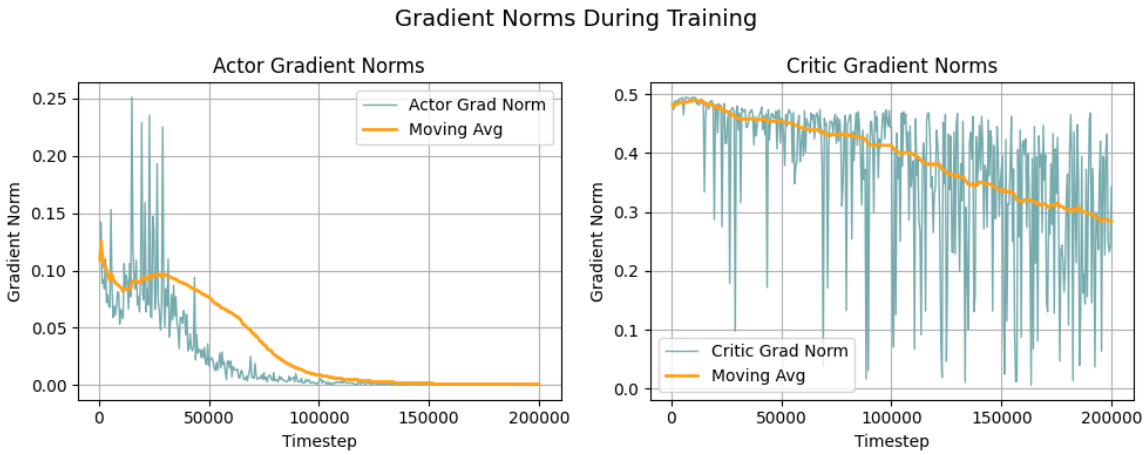


Figure 5.8: Actor and Critic gradient norms during training.

Gradient norms and loss values reveal very important information about the experiments. The actor's gradient norms remain an order of magnitude smaller than the critic's, indicating a severe imbalance in learning signals. Notably, the actor's gradient completely vanishes after ten thousand time steps. Although not ideal, these results prove relative feasibility in the purpose of recommending LeetCode problems to learners.

5.2 Application Development

This section aims to thoroughly explain the features of the application, a2cl, and also provide context on the application's lifecycle. In the next sub-sections, we present a comprehensive breakdown of the system architecture. We then demonstrate the design's adherence to development principles that ensure maintainability and scalability. After that, we describe both the frontend and backend modules. Finally, the testing suite and more security features will be explained.

5.2.1 Feature Specification

This software product, named a2cl, is designed to offer a solution for users looking to track and improve upon their LeetCode objectives. The interface is divided into 3 main components.

- **The Main Screen:** here, users can inspect or delete their previously created plans or choose to create new plans.
- **The Plan Screen:** on this screen, users can examine each plan in detail. The problems belonging to the plan take on a graph structure, made to increase clarity.

- **The View Plans Screen:** users can review each of the plans in more detail than on the main screen. Each plan will have its completion percentage, title, and corresponding topics all clearly shown.

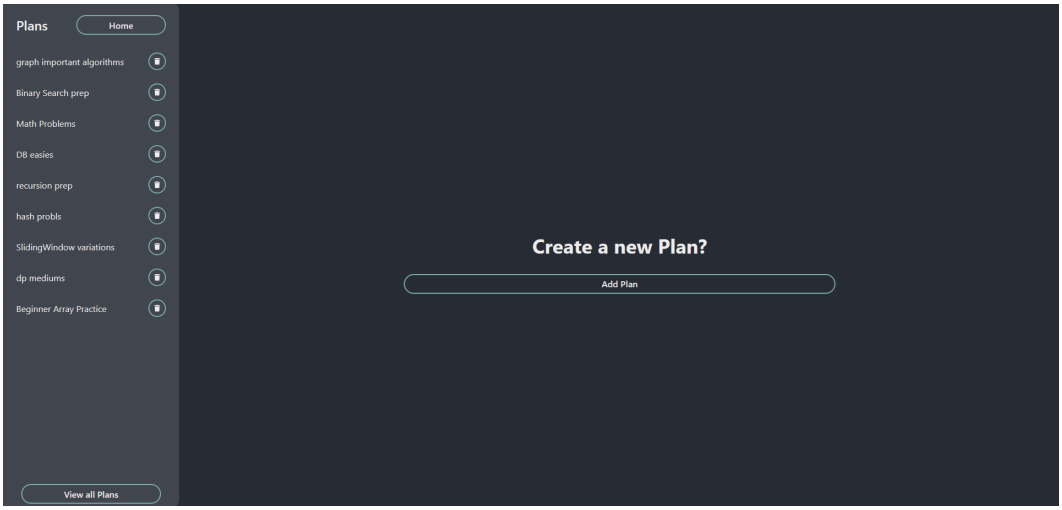


Figure 5.9: A2cl's main screen user interface.

The left side-panel provides a summary of all available plans, where each plan's title is displayed. Below is a detailed breakdown of all the attributes defining a plan:

- **Id:** A unique integer identifier automatically assigned upon creation. This identifier is the primary key for all database operations and API requests.
- **Title:** A name for the plan that follows strict formatting rules, to be revealed in the last sub-section.
- **Profile:** An associated learning profile that the plan corresponds to. This field can correspond to an already existing or a completely new profile.
- **Problems:** A collection of all coding problems included in the plan.
- **GivenTopics:** The target topics. These are the topics given by the user that guide plan generation.

On the Main Screen, users can fully manage their learning plans. Each plan entity is clickable (selecting a plan will open it for detailed viewing and editing). Also, delete icons are provided for quick and intuitive plan removal. Additionally, users can transition to the View Plans Screen and Add Plan screen using the corresponding buttons.



Figure 5.10: A2cl's plan screen user interface.

The Plan Screen provides a simple interface for interacting with each coding problem corresponding to a plan. Each problem can be marked as complete or incomplete. The problem graph, made using the ReactFlow library [Gmb], is essential to the presentation of each plan. Users can freely navigate this screen using click-and-drag movements. The graph adapts to any screen display size, from mobile devices (320px) to desktop monitors (1920px).

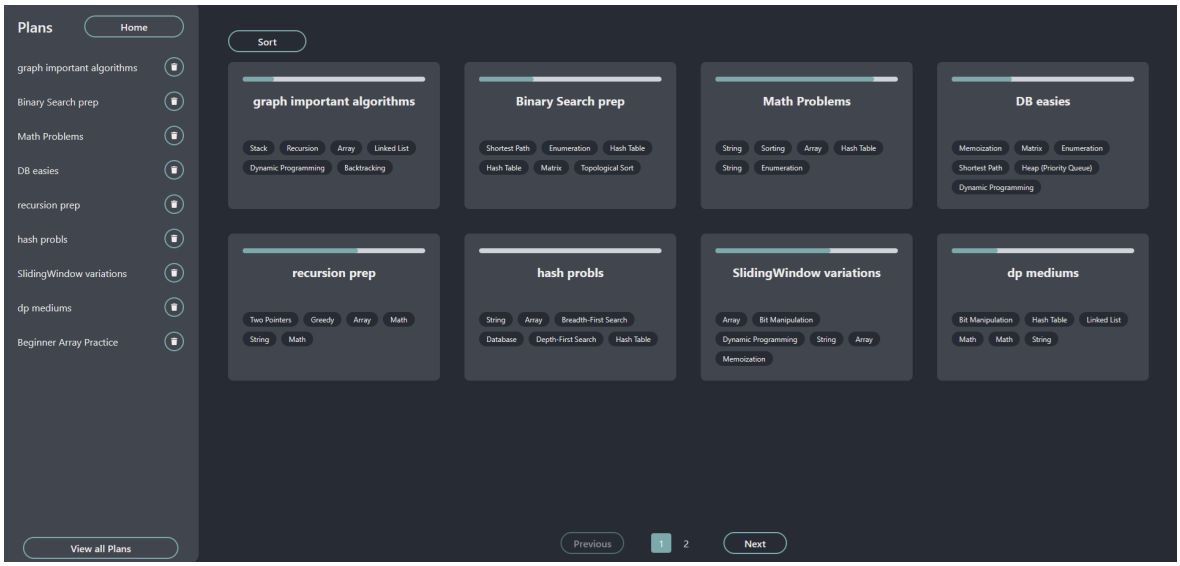


Figure 5.11: A2cl's view plans screen user interface.

The View Plans Screen presents a compact but detailed overview of all available plans. Each plan appears as a card element that displays: the plan title, progress indicator, and topic tags associated with said plan. This screen shows up to eight cards per page, balancing density with readability. Each card component also func-

tions as a navigation element. Clicking any card transitions to the dedicated Plan Screen. Additionally, plans can be locally sorted alphabetically by plan name.

5.2.2 Architectural Overview

The application is built on a three-layer architecture. The frontend is developed using Next.js, a React-based framework, the backend is implemented with FastAPI and is responsible for hosting the reinforcement learning (RL) algorithm and environment, managing business logic, and interacting, via SqlAlchemy ORM [Bay24], with the data layer. This data layer uses a PostgreSQL relational database to store information and track progress. Below we present a very brief diagram of the architecture.

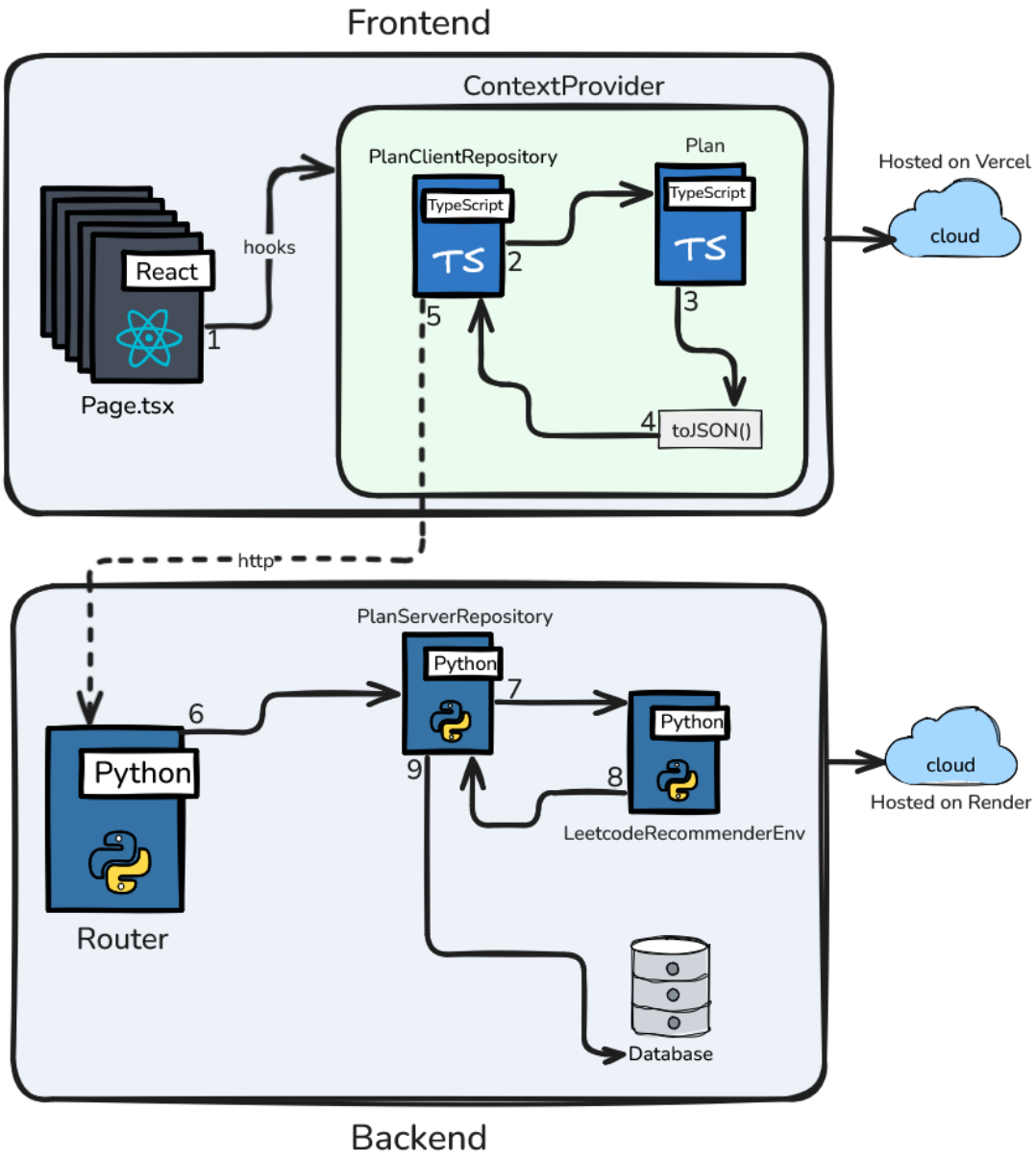


Figure 5.12: A2cl's frontend and backend architecture diagram.

5.2.3 Frontend Architecture and User Experience

The frontend is built around Next.js and React, using server-side rendering for performance and a component-based design. Styling is done using TailwindCSS. The frontend is written in TypeScript, a statically typed superset of JavaScript. This reduces runtime errors and improves overall code quality. Additionally, we take advantage of React's highly reusable components with `PlanCard`, `ProgressBar`, `PlanList`, and `TopicTags` being reused multiple times both in the Main Screen and View Plans Screen.

The graph structure used to display each problem in the Plan Screen is built using the ReactFlow library. The implementation features reusable graph node and edge components (`ProblemEdge` and `ProblemNode`). Each node is color-coded by difficulty: red for hard, orange for medium, and green for easy.

These pages interact with `PlanContextProvider`'s state using a custom hook named `usePlans` which uses the `useContext` React hook. The aforementioned context provider serves as a local cache of the most recent interactions with the backend. Within this provider, the `PlanClientRepository` handles all communication with the backend, both sending and processing incoming requests.

The front-end application is hosted on Vercel, a cloud platform optimized for Next.js and React applications. Vercel automatically builds and deploys every Git branch as a preview environment. Each deployment includes real-time logs for debugging.

5.2.4 Backend Architecture and Data Flow

The backend is built with FastAPI, serving as the interface between the frontend and the system. It exposes RESTful API endpoints that handle requests from the frontend and ensures type-safe communication through Pydantic [L+24] schemas. These schemas are automatically validating requests across all entities.

Table 5.4: PlanJSON Schema Definition

PlanJSON Schema	
`id`	`int`
`title`	`str`
`profile`	`ProfileJSON`
`problems`	`List[Tuple[bool, ProblemJSON]]`
`topics`	`Optional[List[TopicJSON]]`
`given_topics`	`List[TopicJSON]`

Beneath the API layer, we use SQLAlchemy's ORM to manage all the database operations, therefore abstracting interactions with the PostgreSQL database. Mod-

els defined in SQLAlchemy map directly to database tables. This system also uses a seeding mechanism built alongside SQLAlchemy to populate the PostgreSQL-database with all existing leetcode problems and topics. These seeders operate under two conditions: as a one-time initialization step during production deployment, or as an automatic repopulation mechanism during development whenever database tables are empty. The backend server and database server are both hosted on Render.

The reinforcement learning (RL) environment operates on the backend. When handling a request, the model first checks if an existing environment with the same input (target topics) has already been saved. If not, a new one is created. Next, the model runs five trajectories (episodes) and selects the one with the best reward. The environment stores the IDs of the recommended problems and sends a copy of them back to the `PlanServerRepository`.

During hyperparameter tuning, logs were used to monitor many different statistics about the environment. Checkpoints of each model were extracted as well. These features were implemented using Stable-Baselines3's [RHG+21] callback system that allows access to the model's internal state during training or testing. These callbacks are divided into two main categories: statistics tracked each episode and statistics tracked every 500 time steps. Below, we describe all of them and also detail an implementation of a callback:

- **Rewards per Episode + moving average:** assesses the agent's performance.
- **Timesteps per Episode:** tracks episode duration. Short duration indicates early stopping and longer durations indicate inefficient exploration.
- **Action Entropy per Episode + moving average:** measures exploration versus exploitation. High entropy denoting almost random behavior and low entropy denoting deterministic behavior.
- **Actor Loss per time-step + moving average:** assesses how well the policy is improving. Steadily decreasing loss would show desirable behavior while spikes or instability denotes training issues.
- **Critic Loss per time-step + moving average:** tracks the improvement of the critic. Very similar with the previous statistic.
- **Actor gradient norm per time-step + moving average:** tracks magnitude of the policy gradient updates. Large norms mean aggressive updates and low norms mean slow convergence.

- **Critic gradient norm per time-step + moving average:** tracks magnitude of the value function gradient updates. Both of these are calculated using the L2 Norm.

Algorithm 3 on_step

Collect reward r_t and action a_t into variables
if environment is done or truncated **then**
    Collect r_t to list
    Compute reward moving average $\bar{r}$
    Collect $\bar{r}$ to list
    Compute entropy H and its moving average $\overline{H}$
    Collect $\overline{H}$ to list
    Write to file and flush
end if
if $t \bmod 500 = 0$ **then**
    Get actor and critic losses: $\mathcal{L}_{\text{actor}}, \mathcal{L}_{\text{critic}}$
    Collect $\mathcal{L}_{\text{actor}}, \mathcal{L}_{\text{critic}}$ to lists
    Compute losses' moving averages
    Compute gradient norms $\|\nabla\mathcal{L}_{\text{actor}}\|$, $\|\nabla\mathcal{L}_{\text{critic}}\|$
    Collect norms to lists
    Compute norms' moving averages
    Write to file and flush
end if

5.2.5 Security, Validation and Testing

A2cl employs some security measures to protect data integrity and system reliability. These measures are: encrypted communication using the HTTPS protocol and database query handling using prepared statements [Pos24]. These strategies mitigate many common attack vectors.

Client-side API requests are managed using HTTPS. This prevents man-in-the-middle attacks and protects sensitive data. On the backend, even if a request is made using an unsecured connection, it will be upgraded to a secure channel before continuing.

Database queries are handled by SQLAlchemy's ORM. The most important columns of a table, such as email, password hashes, and identifiers, are commonly targeted in SQL injection attempts. To defend against this, a2cl uses parameterized queries on all indexed columns.

We implemented a validation system that imposes data integrity on all domain

entities. This validation happens both in the backend and in the frontend, ensuring consistency throughout the application. Both `PlanValidator` and `ProfileValidator` validate all fields of their corresponding classes. All fields are checked against their expected data types. Additionally, strings are checked against length constraints (minimum, maximum, and empty) and special characters. Integers are verified against their expected ranges.

We conducted a series of tests on our backend implementation. These tests include unit tests for individual components and integration tests to verify the interaction between different modules.

- **Unit Testing**
  - **Implementation Framework**: The test suite was created using Pytest, chosen mostly for its simplicity. The framework's fixture functionality was utilized a lot to create reusable test components.
  - **Domain Entity Coverage**: Every domain entity was tested with all possible decision paths verified, getting 100% decision coverage.
  - **Repository Layer Verification**: The tests validated all CRUD operations and business logic interactions with the repository layer. The stopping criterion was also getting 100% decision coverage.
  - **Serialization Testing**: Tests verified that all entity's fields were correctly transformed to a JSON format correctly and that the serialization and deserialization preserved data integrity.
- **Integration Testing**
  - **API Endpoint Validation**: Each REST endpoint was tested with these HTTP methods: GET, POST, PATCH and, DELETE.

Chapter 6

Conclusion and Future Work

This thesis makes a contribution to the field of reinforcement learning (RL) by developing the first LeetCode problem recommender system. The main innovation lies in framing personalized problem recommendations as a sequential decision-making task, which addresses a gap in current educational tools. By using user-specific learning targets and statistical problem data, the proposed algorithm demonstrates relative feasibility of automating problem recommendations.

Empirical results indicate steady convergence, the reward function shows good improvement (rising from an initial value of 100 to having over 400 moving average after only 5000 episodes) while entropy values consistently declined, settling around 3 by the end of training. These trends suggest that the agent was exploring the problem space and refining its policy efficiently. However, a constant challenge was the actor network's vanishing gradient. This heavily limited the model's ability to run for more episodes and improve the recommendations. Despite many attempts to solve this issue (gradient clipping, multiple weight initialization methods, simpler and more complex network architectures, different coefficient values for losses, different learning rates, different gradient descent optimizers, and using multiple feature extractors), the problem is still unresolved. Future work should investigate alternative methods to address this issue.

However, the empirical results presented in Chapter 4 indicate that the system's recommendations could not have fully achieved the desired realism. The algorithm converged slowly and iteratively improved throughout training, but yet, generating realistic recommendations proved insufficient without highly complex embeddings [ZDC+20] or realistic user simulations [ZZX+19] (or both [ZXD+20]). Additionally, the extreme topic imbalance in the dataset (where over half of the topics had the "Array" tag and more than 40 topics had under 10 representatives) heavily skewed recommendations toward dominant categories.

Looking forward, several strategic directions will elevate this work. Firstly, developing a model-based RL environment with many parallelized simulations will

more efficiently mimic real user interactions. Also, making sparse use of online algorithms to gather significant context via limited real-user feedback would drastically improve reliability. Secondly, generating rich embeddings for each LeetCode question, by using knowledge graphs (KGs) and graph neural networks (GNNs) holds much promise. However, constructing LeetCode-specific KGs requires significant effort, which was well past the scope of our work.

In summary, this research demonstrates the viability of RL-driven recommendation systems for problem-solving platforms. Despite many current limitations in data balance and model optimization, the framework provides an acceptable foundation. Future work incorporating advanced simulations, online learning, and KG-enhanced embeddings provide a clear trajectory toward real-world deployment.

Bibliography

[ALP16] Adnan Aziz, Tsung-Hsien Lee, and Amit Prakash. *Elements of Programming Interviews in Python*. CreateSpace Independent Publishing Platform, 2nd edition, 2016.

[Bay24] Michael Bayer. Sqlalchemy: The database toolkit for python. `https://www.sqlalchemy.org`, 2024.

[Far22] Foundation Farama. Gymnasium: Api standard for reinforcement learning. `https://gymnasium.farama.org/index.html`, 2022.

[Gmb] Webkid GmbH. Reactflow. `https://reactflow.dev`.

[Goo23] Google. Google trends. `https://trends.google.com/`, 2023.

[Gro18] Alibaba Group. Alibaba large-scale behavior data. `https://tianchi.aliyun.com/dataset/dataDetail?dataId=649`, 2018.

[HK15] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):19:1–19:19, 2015.

[JK02] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.

[L+24] Samuel Lancaster et al. Pydantic: Data validation using python type hints. `https://docs.pydantic.dev`, 2024.

[Lau25] Dan Laurentiu. A2cl: Advantage actor-critic. `https://github.com/DanLaurentiu1/a2cl`, 2025.

[Lee15] LeetCode. Leetcode: The world's leading online programming learning platform. `https://leetcode.com/`, 2015.

[MBM+16] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P. Lillicrap, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *arXiv preprint arXiv:1602.01783*, 2016.

[Nee22] NeetCode. Neetcode 150: Coding interview study plan. `https://neetcode.io`, 2022.

[Ope17] OpenAI. Baselines: Acktr & a2c, 2017.

[Pos24] PostgreSQL Global Development Group. *PostgreSQL Documentation: PREPARE*, 2024.

[Ret16] RetailRocket. Retailrocket recommender system dataset. `https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset`, 2016.

[RHG+21] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Noah Ernestus, and Raphaël Dormann. Stable-baselines3: Reliable reinforcement learning implementations. `https://github.com/DLR-RM/stable-baselines3`, 2021.

[SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[Sil15] David Silver. Reinforcement learning course (compm050/compgi13). University College London (UCL) Lecture Slides, 2015.

[Ski53] B.F. Skinner. *Science and Human Behavior*. Simon and Schuster, 1953.

[SML+15] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[SMSM00] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems (NeurIPS)*, 12:1057–1063, 2000.

[TS06] Andrew Turpin and Falk Scholer. User performance versus precision measures for simple search tasks. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 11–18. ACM, 2006.

[ZDC+20] Sijin Zhou, Xinyi Dai, Haokun Chen, Weinan Zhang, Kan Ren, Ruiming Tang, Xiuqiang He, and Yong Yu. Interactive recommender system via knowledge graph-enhanced reinforcement learning. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 29–38. ACM, 2020.

[ZMKL05] Cai-Nicolas Ziegler, Sean M McNee, Joseph A Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th international conference on World Wide Web (WWW)*, pages 22–32. ACM, 2005.

[ZXD+20] Lixin Zou, Long Xia, Pan Du, Zhuo Zhang, Ting Bai, Weidong Liu, Jian-Yun Nie, and Dawei Yin. Pseudo dyna-q: A reinforcement learning framework for interactive recommendation. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 665–673. ACM, 2020.

[ZZX+19] Xiangyu Zhao, Liang Zhang, Long Xia, Zhuoye Ding, Dawei Yin, and Jiliang Tang. Deep reinforcement learning for list-wise recommendations. In *Proceedings of the 1st Workshop on Deep Reinforcement Learning for Knowledge Discovery (DRL4KDD)*. ACM, 2019.